The Agent Is a Process: A Token-Fair Runtime for Multi-Agent LLM Systems

Atharva Arbat

July 2026

Multi-agent LLM systems are deployed as applications but governed as none. Concurrent agents contend for token throughput, API dollars, and provider rate-limit slots with no allocation policy beyond arrival order: a runaway agent starves its siblings, priority classes exist only in application logic, and per-agent spend is discovered after the fact on an invoice. Serving-side work has established fair scheduling *inside* inference engines (Sheng et al. 2024), where the quantum is a preemptible token-granular batch and the engine observes output tokens as they are produced. We address a structurally different problem one layer up, at the client runtime, where a single agent may fan out across several providers, models, and non-model tools at once. Here the quantum is an entire API call; it is non-preemptible once dispatched; and its cost is unknown until it completes. No provider can schedule this workload, because no provider sees all of it.

Our contribution is a scheduler for this regime and a proof of its fairness. Reaching it requires the right interface, so we argue that the agent is a **process**, not a chat session: the process is the abstraction under which fair scheduling is even *statable*, because it supplies the per-agent budget, the scheduler state, and the single mediated boundary the scheduler operates on. We build **AgentKernel** on one invariant, *complete mediation*—every model call, tool invocation, inter-agent message, clock read, and random draw passes through the kernel—and prove that no non-mediating layer, framework middleware in particular, can host the scheduler at all (Proposition 1). On that substrate we extend virtual-time fair queueing to non-preemptible, unknown-cost quanta by charging an *estimated* cost at dispatch and *reconciling* the actual cost at completion. We call the

discipline **Reconciled Virtual Time (RVT)** and prove a service-gap bound of $d\,(C_{\max}/w_i + \hat{E}_{\max}/w_j)$ between any two continuously backlogged agents (Theorem 1), which specializes to $2C_{\max} + E_{\max}$ under an estimator-error parameterization (Corollary 1) and yields starvation freedom as a corollary. Two negative results delimit the design space: without reconciliation the service gap is unbounded in time (Proposition 2), and no non-mediating layer can enforce the premises the bound assumes (Proposition 1). As a secondary result we adapt Dominant Resource Fairness (Ghodsi et al. 2011) to the agent resource vector (tokens/min, requests/min, dollars/epoch), degrading to the scalar bound when one resource dominates.

We evaluate on a deterministic mock-LLM harness calibrated from real API traces, driving up to 50 concurrent agents under skewed demand, adversarial bursts, and heavy-tailed output-length distributions, with small-scale live runs against two production providers to validate mock fidelity. [TBD: headline fairness, service-gap, and overhead numbers.]

1. Introduction

Five agents share one API budget and one impatient human. One of them decides to summarize a two-hundred-page document and issues a 100K-token call. For the next thirty seconds the other four get nothing: the provider's rate-limit window is full, and nothing in the system is arbitrating. Whichever agent's HTTP request happened to be in flight first has won, and the human waiting on the interactive agent is paying for that accident.

This is a scheduling failure, and it is the failure this paper removes. It is worth being precise about why it persists. The dominant multi-agent frameworks—LangGraph (LangChain 2024), AutoGen (Wu et al. 2023), CrewAI (CrewAI 2024), MetaGPT (Hong et al. 2024), CAMEL (Li et al. 2023)—are orchestration libraries. They define *what* agents say to each other: the graph, the handoff protocol, the role prompts. None of them defines *how the system's scarce resources are allocated* when those agents run at the same time. The result is a class of systems that have concurrency without a concurrency policy.

1.1. The deficit

Three symptoms recur across deployments.

Monopolization. A single agent issuing large or frequent calls saturates the provider's rate-limit window and stalls every sibling for the remainder of the epoch. There is no notion of a share, so there is no notion of exceeding one.

Unenforced priority. Applications routinely distinguish a user-facing agent from a background one, but the distinction lives in comments and naming conventions. Nothing at the infrastructure layer dispatches the interactive agent first, and nothing prevents the background agent from consuming the window.

Retroactive accounting. Dollar spend is metered by the provider after the fact. No runtime enforces a per-agent budget, so a looping agent's cost is discovered when the bill arrives rather than when the loop starts.

The resource vector is itself unusual. Agents contend simultaneously over *tokens per minute* (the provider's throughput limit), *requests per minute* (a separate concurrency limit), and *dollars per epoch* (a cost budget). These dimensions are not proportional to one another. An agent issuing many short calls bottlenecks on requests/min while an agent issuing a few long ones bottlenecks on tokens/min; a third, calling an expensive reasoning model rarely, bottlenecks on dollars. Scalar fair share is not merely hard to compute in this setting—it is ill-defined, because the agents are not competing for the same scalar.

1.2. Why the runtime is the right vantage point

One could ask the providers to be fair, and for a single model behind a single API, VTC (Sheng et al. 2024) shows how. But an agent system is not a single model behind a single

API. In a realistic deployment, Agent A reasons with GPT-4 while Agent B reasons with Claude; Agent C's spend is dominated by a web-search tool and Agent D's by a database it queries repeatedly, so that for those two the model calls are a minority of the cost; and all four share one session budget in dollars and one human's patience in latency.

No provider can see this picture. OpenAI's scheduler sees Agent A's calls and nothing of B, C, D, the search tool, or the database. Anthropic's scheduler sees the mirror image. The search API sees neither. **The only vantage point from which the whole resource contention is visible is the runtime the agents share.** Fairness, budgets, and priority are therefore properties that can only be enforced *above* the provider, across heterogeneous back-ends, and over both model and non-model resources.

Figure 1 makes the vantage point literal. Every back-end—two providers and two external tools—crosses one boundary, and only the runtime that owns that boundary sees them all at once. This is what makes client-side scheduling a distinct and necessary problem rather than a redundant copy of serving-side scheduling.

1.3. Why existing scheduling theory does not transfer

VTC (Sheng et al. 2024) proves a 2× service bound for LLM serving and is the closest prior work. It operates inside the serving engine, where continuous batching (Yu et al. 2022) permits preemption at token granularity and the engine observes output tokens as they are generated. At the client runtime, three properties invert.

The quantum is an entire call. Once dispatched, an `infer` cannot be preempted mid-token. The scheduler's only mid-call primitive is `cancel`, which truncates and forfeits the work done so far.

Cost is unknown at dispatch. The scheduler must commit to a dispatch order before knowing how long the call will run or how much it will cost, because output length—the dominant cost term—is revealed only on completion.

The scheduler is above the provider API. It cannot interpose on token generation to reclaim the resource early, and it cannot see the queueing that happens on the far side of the boundary.

This is closer to packet scheduling with unknown packet sizes over a constrained link—a regime classical fair queueing explicitly excludes, since WFQ (Demers, Keshav, and Shenker 1989) and SFQ (Goyal, Vin, and Cheng 1996) both assume the packet size is known at enqueue, as do lottery and stride scheduling (Waldspurger 1994; Waldspurger and Weihl 1995). Worse, unlike a router, we cannot drop or fragment. A fairness bound for this regime must account for both the non-preemptibility gap—one long call holding the resource past its fair quota—and the error introduced by predicting cost. Neither VTC's analysis nor the classical fair-queueing analyses cover it.

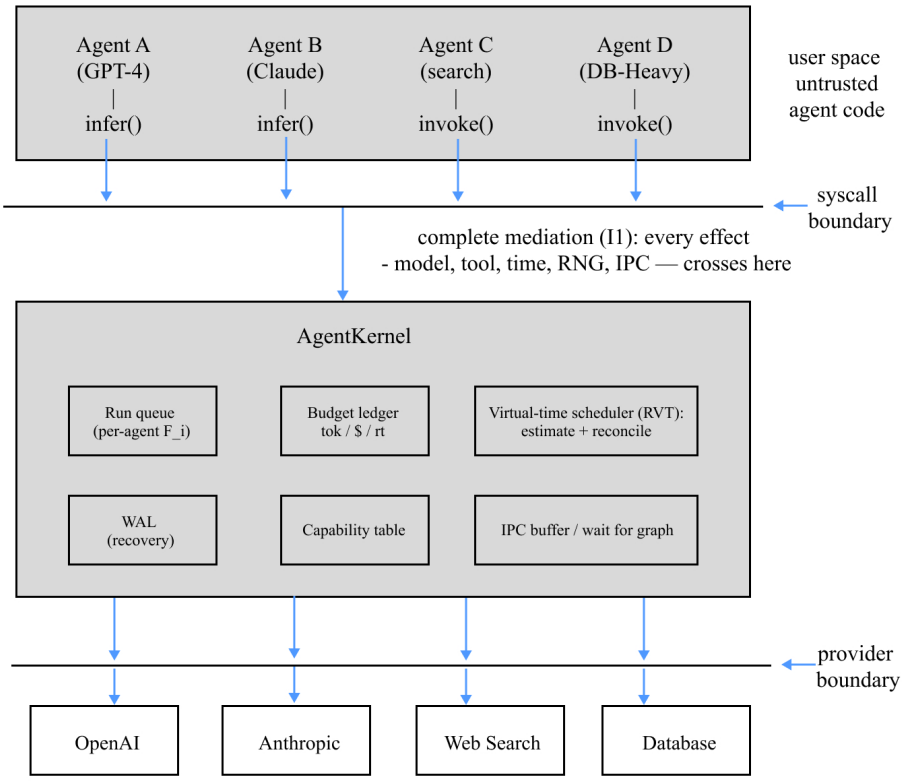

FIGURE 1. The complete-mediation boundary. Four agents, each with a different dominant back-end, issue `infer` and `invoke` syscalls that cross a single boundary into AgentKernel; the kernel's run queue, budget ledger, virtual-time scheduler, write-ahead log, capability table, and IPC buffer sit above a second boundary, below which each provider or tool sees only its own column. No back-end below the provider boundary sees more than one column; only the kernel sees every column, which is why fair scheduling is enforceable here and nowhere else (§1.2, §3.3).

1.4. Thesis

The process abstraction, not the chat abstraction, is the correct systems interface for autonomous LLM execution. An LLM agent is a process: it has a private context region, a resource budget, scheduler state, message channels, and a lifecycle. If a runtime enforces complete mediation—the agent never touches the model, tools, time, randomness, or other agents except through the kernel—then proportional-share scheduling becomes implementable with classical virtual-time machinery, adapted for non-preemptible, unknown-cost quanta.

Complete mediation is not merely convenient for this; §3.3 proves it necessary. The scheduler that results, Reconciled Virtual Time, is the paper's central result. The process abstraction earns its place in the paper by being what makes that result statable and provable.

1.5. Contributions

1. **Reconciled Virtual Time (RVT)** (§4): a proportional-share discipline for non-preemptible, cost-unknown-until-completion quanta, with a proven service-gap bound (Theorem 1), starvation freedom (Corollary 2), and a matching lower bound (Proposition 3). The analysis isolates *reconciliation* as the mechanism that bounds unfairness and shows that the estimator affects the bound only through the magnitude of the in-flight charge, not through its accuracy—a result that revises the intuition that better prediction buys fairness.
2. **Two negative results** delimiting the design space: without reconciliation the service gap grows without bound in time (Proposition 2), and no non-mediating layer—framework middleware in particular—can enforce the premises the bound requires (Proposition 1).
3. **The agent-process abstraction that makes RVT statable** (§3): a field-by-field process control block for agents, a fixed syscall surface, five invariants, and a stratification of execution forms by how completely each closes the bypasses Proposition 1 enumerates.
4. **An implemented runtime and evaluation harness** (§6, §7): AgentKernel—syscall interface, budget ledger, write-ahead log, and two enforced execution forms—together with a deterministic mock-LLM harness calibrated from real traces and an adversarial suite for estimator and mediation stress.

2. Problem Statement and System Model

We fix notation used throughout.

Agents and weights. A set of agent processes $i = 1 \dots n$ share one kernel. Agent i carries a positive weight w_i expressing its entitled share; equal weights give equal shares. Weights are administrative input, not inferred from behavior.

Calls and cost. Agent i submits calls c through the syscall interface (§3.2). Following the VTC cost form (Sheng et al. 2024), an `infer` call has cost

$$\text{cost}(c) = \alpha \cdot \text{in\_tokens}(c) + \beta \cdot \text{out\_tokens}(c),$$

where $\alpha, \beta > 0$ are provider- and model-specific coefficients. Non-model `invoke` calls carry a cost in the same units, either metered by the tool or assigned by policy. We write

$a(c)$ for the *actual* cost, revealed only at completion, and $\hat{e}(c)$ for the kernel's *estimate*, computed at dispatch. Crucially, in_tokens(c) is known at dispatch and out_tokens(c) is not, so the uncertainty is confined to the β term but is not thereby small: output length is the heavy-tailed component.

Non-preemptibility. Once a call is dispatched to a provider it runs to completion. The only intervention available is `cancel`, which truncates the call; the tokens already generated are still charged, so cancellation forfeits work rather than reclaiming it.

Concurrency. The kernel dispatches from a single logical loop, so dispatch decisions are totally ordered (this also makes them loggable and deterministic; §3.4, I4). Each agent may have at most d calls in flight simultaneously, a kernel-configured bound. The natural setting for a sequential agent is $d = 1$: the agent blocks on its `infer` and cannot issue another until it returns. Fan-out agents run with $d > 1$.

Constraints. Dispatch is subject to provider-side admission constraints—requests per minute, tokens per minute—and to per-agent budget constraints in tokens and dollars. We write admissible(i) for the predicate that agent i's next call may be issued now without violating either.

Service. Define $A_i(t)$, the *reconciled service* of agent i, as the sum of actual costs $a(c)$ over all of i's calls completed by time t. This is the quantity fairness is about: what the agent actually consumed, not what it was predicted to consume.

Objectives. The scheduler must be (a) *fair*: bound $|A_i(t)/w_i - A_j(t)/w_j|$ for continuously backlogged i, j; (b) *starvation-free*: every backlogged agent is dispatched within bounded work, including under priority classes; (c) *work-conserving*: never idle while some backlogged agent is admissible; and (d) *deterministic*: given the log prefix, the dispatch sequence is a function of logged state.

Bounded quantities. We assume $a(c) \le C_{\max}$ and $\hat{e}(c) \le \hat{E}_{\max}$ for all calls, and $a(c) \ge c_{\min} > 0$. The first holds because providers cap `max_tokens`; the second because the kernel controls the estimator; the third because every call consumes at least its prompt.

3. The Agent-Process Abstraction

The title claims an agent is a process. This section discharges that claim: it reconstructs the process control block for an agent (§3.1), defines the syscall interface that gives the abstraction teeth (§3.2), proves that only a mediating kernel can host it (§3.3), states the invariants the kernel enforces (§3.4), and stratifies the execution forms by the assurance each provides (§3.5). We build the abstraction here not for its own sake but because §4's scheduler cannot be *stated* without it: the budget vector, the per-agent scheduler state, and the single mediated boundary are precisely the preconditions Theorem 1 assumes.

TABLE 1. Classical PCB fields and their agent-process analogues

Classical PCB field	Agent-process analogue	Enables
Address space / page tables	Context region C (per-agent working set)	Isolation of one agent's working set from another's
Registers / program counter	Conversation state + pending tool calls	Resumption after a blocking syscall
Scheduler state (nice, vruntime)	Virtual time F_i, weight w_i, priority π	Proportional-share dispatch (§4)
Resource limits (`rlimit`)	Budget vector $B = (B_{\text{tok}}, B_{\$}, B_{\text{rate}})$	Per-agent budget enforcement
File descriptor table	Tool handles / capabilities	Least-authority tool access
Signals	`cancel` (preemption by truncation)	The only mid-call preemption primitive available
IPC (pipes, message queues)	`send`/`recv` + kernel message buffer	Inter-agent coordination the kernel can observe
Process state {run, ready, blocked}	{`created`, `runnable`, `running`, `blocked`, `terminated`}	Lifecycle accounting; wait-for graph

3.1. The process, reconstructed

A classical process is not its code; it is the kernel's bookkeeping *about* that code—the process control block. An agent admits the same bookkeeping, field for field. The correspondence below is not an analogy for rhetorical effect: each row names a concrete data structure the kernel maintains and the concrete mechanism it enables.

DEFINITION 1 (Agent process). *An agent process is a tuple* $P = (id, S, C, B, \pi, L)$ *where* $S \in \{\texttt{created}, \texttt{runnable}, \texttt{running}, \texttt{blocked}, \texttt{terminated}\}$ *is the lifecycle state;* C *is the private context region;* $B = (B_{\text{tok}}, B_{\$}, B_{\text{rate}})$ *is the multi-resource budget vector;* π *is the priority class; and* L *is the handle into the write-ahead log.*

The single structural difference from a UNIX process is the one that generates this paper's entire technical contribution. **A classical scheduler knows the cost of the quantum it is about to run—one timeslice—and can reclaim the CPU at the end of it with a timer interrupt. The agent scheduler knows neither: the cost of an `infer` is unknown until it returns, and no interrupt can reclaim a call in flight.** Section 4 is the repair of that one break.

3.2. The syscall interface

The abstraction is real only if the agent cannot act except through it. The kernel exposes a small, fixed syscall surface—the only channel between an agent and the world:

`infer` (model call), `cancel` (abort an in-flight call), `invoke` (tool call), `send`/`recv` (inter-agent messages), `clock`, `random`, `spawn`, `exit`.

Every syscall is scheduled and logged. Each tool handle passed to `invoke` is an explicit capability rather than an ambient permission (§3.4, I2); this paper relies on that only insofar as the scheduler must attribute every tool call to an agent, and leaves capability enforcement and its security analysis out of scope. `send` is asynchronous and kernel-buffered; `recv` blocks, moving the agent to `blocked`. Because the kernel observes every `recv` wait, it maintains the inter-agent wait-for graph, which is what makes deadlock detection and cross-agent priority inheritance possible (§5.2)—a scheduling dividend of mediation rather than a separate subsystem.

Figure 1 shows this surface concretely: every agent arrow terminates at the syscall boundary, and nothing reaches a provider except through the kernel sitting on it.

3.3. Why a kernel, and not middleware

A reviewer will reasonably ask why the scheduler needs a privileged kernel rather than cooperative middleware: a framework could intercept LLM-client calls as a library shim. The answer is that the scheduler depends on properties a non-mediating layer cannot supply. We state it as a proposition because it is the load-bearing justification for the architecture.

PROPOSITION 1 (Necessity of complete mediation). *Let a scheduler require, for its fairness and starvation guarantees, that (P1) every resource-consuming effect is* charged *to an agent before it takes effect; (P2) effects are* serialized *through a single dispatch decision the scheduler controls; and (P3) every inter-agent wait is* observable *to the scheduler. Then any layer M that does not mediate all of* {`infer`, `invoke`, `send`/`recv`, `clock`, `random`} *cannot guarantee P1–P3.*

PROOF. Suppose M leaves some syscall unmediated. We show each case defeats at least one required property.

Clock. If M does not mediate `clock`, an agent reads wall-clock time directly and can busy-wait, poll, or time its own bursts outside the scheduler's frame. The scheduler cannot serialize activity it never observes, defeating P2.

Model and tool calls. If M does not mediate every `infer` and `invoke`, an agent can construct a model or tool client directly—a raw socket, a second SDK instance—and consume tokens and dollars never charged to the ledger. The sum of charged costs then diverges from metered usage, defeating P1 (and with it invariant I5, on which Theorem 1's identification of A_i with real consumption depends).

IPC. If M does not mediate `send`/`recv`, agents coordinate through a side channel—a shared file, an external message bus—and block on one another invisibly. The wait-for graph is then incomplete, so priority inheritance and deadlock detection are unsound, defeating P3.

Ordering. If any effect reaches a provider without passing the dispatch point, the scheduler's serialization is advisory rather than enforced: a backlogged agent jumps the queue simply by not calling the mediated path, defeating P2.

Randomness. If M does not mediate `random`, agent behavior—including retry and backoff timing, which consumes rate-limit slots—is neither reproducible nor accountable, so the scheduler cannot bound the contention it is meant to control.

Each bypass is available to any layer that agents can go *around*. A layer can be gone around exactly when it does not sit on the sole path to the effect—that is, when it is not a mediating kernel. Hence complete mediation over {`infer`, `invoke`, `send`/`recv`, `clock`, `random`} is necessary for P1–P3. □

The corollary is architectural: middleware can *approximate* the scheduler but cannot *guarantee* it, because an agent—or a bug—can always construct the underlying client itself. This is why §3.5 stratifies execution forms by how strongly each closes the bypasses, and why we measure shim leakage empirically (§7.4) rather than assuming it away.

3.4. Invariants

The kernel enforces five invariants. This paper proves the scheduling consequences of I1, I4, and I5; I2 and I3 are part of the kernel's design and are stated here to fix the interface, but their security and recovery implications are beyond a scheduling paper's scope.

I1 (complete mediation (Saltzer and Schroeder 1975)). No agent-visible or agent-external effect occurs except via a syscall. Proposition 1 shows I1 is necessary for the scheduler's guarantees; §4 shows it is sufficient.

I2 (no ambient authority (Dennis and Horn 1966; Miller 2006)). Every `invoke` carries an explicit capability rather than relying on the agent's ambient permissions. The scheduler uses this only to attribute each tool call to an agent.

I3 (log-before-effect (Mohan et al. 1992)). Every nondeterministic input is durably appended to the write-ahead log before the agent observes it. Here the log serves crash recovery; we do not exploit it for deterministic replay.

I4 (deterministic kernel). Given the log prefix, every kernel decision is a deterministic function of logged state. This constrains the scheduler concretely: ties in F_i must be broken deterministically, and the dispatch order is itself a logged event.

I5 (budget conservation). Ledger accounting is exact: the sum of per-agent charged costs equals metered provider usage within reconciliation error. Theorem 1 bounds a gap in *charged* service; I5 is what makes that a statement about real consumption.

3.5. Enforcing I1: agent execution forms

I1 is a claim about what agent code *cannot* do, so it is only as strong as the environment hosting that code. Classical kernels get user/kernel separation from a hardware privilege bit; we must buy it. AgentKernel defines two supported forms, ordered by how completely they close the Proposition 1 bypasses.

SDK agents (cooperative mediation). Agents are written against the kernel's syscall SDK; mediation is by convention plus static lint that rejects network, clock, and RNG imports in agent packages. Assurance is "the developer did not bypass"—sufficient for the scheduling and budget-conservation claims, insufficient against agent code that deliberately bypasses the SDK.

Sandboxed agents (enforced mediation). Agent code runs in a WASM module (Haas et al. 2017) or an OS process in a network-denied namespace, whose only host imports are the kernel syscall shims. Here I1 holds by construction against arbitrary agent code: the bypasses are physically unavailable.

For existing framework code, a **compatibility shim** intercepts the LLM- and tool-client call surface and reroutes those calls as syscalls. It is a migration path, not a guarantee; by Proposition 1 it is exactly the leaky case, so we measure its leakage with an adversarial escape suite and report which invariants survive under shim-only mediation (§7.4).

4. Reconciled Virtual Time

We develop the scheduler the way a reader meets the problem: start from the classical machinery, watch it break, and repair it. The theorem arrives only once the need for it is unavoidable.

4.1. Development

Step 0: the guarantee we want. Virtual-time fair queueing (Demers, Keshav, and Shenker 1989; Goyal, Vin, and Cheng 1996), and VTC (Sheng et al. 2024) for serving, gives each backlogged agent i of weight w_i a service share proportional to w_i, with a closed-form bound on how far any two agents' normalized service can drift apart. Each agent carries a virtual time F_i; the scheduler always dispatches the agent with the smallest F_i. This is the substrate we want to keep.

Step 1: where it breaks. The classical bound depends on knowing $\mathrm{cost}(c)$ at enqueue, so that F_i can be advanced correctly at dispatch. For an `infer`, the dominant cost term $\beta \cdot \mathrm{out\_tokens}(c)$ is unknown until completion. The scheduler must pick a dispatch order before it can compute the very quantity that order is supposed to depend on.

Step 2: the naive fix, and why it fails. Charge an estimate $\hat{e}(c)$ at dispatch—say a per-agent EWMA of recent call costs—and advance F_i by $\hat{e}(c)/w_i$. This restores a total order,

but it is not fair. An agent whose calls are *systematically* longer than its estimate advances F_i too slowly, is therefore dispatched too often, and captures more than its weighted share. Proposition 2 below shows the resulting gap is not merely large but unbounded in time. Estimation without correction converts unknown cost into unbounded unfairness.

Step 3: reconciliation. On completion, adjust F_i by the reconciliation delta $(a(c) - \hat{e}(c))/w_i$. An underestimate pushes F_i forward after the fact, throttling the agent's next dispatch; an overestimate refunds it. This estimate-then-reconcile loop is the whole of Reconciled Virtual Time. Figure 2 traces one cycle. Reconciliation guarantees that, integrated over a backlogged period, each agent's virtual time reflects its *actual* consumption; the estimator affects only the timing of when the correction lands, never the total. The residual unfairness is therefore confined to what is still in flight, which is exactly what Theorem 1 captures.

4.2. The algorithm

RVT maintains, per agent i: virtual time F_i, weight w_i, pending estimate mass $P_i = \sum_{c \text{ in flight}} \hat{e}(c)$, in-flight count n_i, reconciled service A_i, and a FIFO queue of submitted calls. B denotes the set of backlogged agents (those with a queued or in-flight call).

Three details carry weight. First, **virtual-time equalization** on rejoining the backlog set prevents an agent that has been idle from accumulating credit and then monopolizing the system to "catch up"—the standard SFQ device (Goyal, Vin, and Cheng 1996), and the reason an agent cannot bank its absence. Second, F_i **is never reset**, not at epoch boundaries and not on budget exhaustion; §4.6 shows that this is exactly what *debt carry-over* amounts to. Third, **tie-breaking is deterministic**, which I4 requires and which also makes the schedule reproducible across runs of the same workload.

4.3. The accounting identity

Everything below rests on one bookkeeping fact.

LEMMA 1 (RVT accounting identity). *Between two consecutive equalization events for agent i, for all t,*

$$w_i\big(F_i(t) - F_i(t_0)\big) \;=\; \big(A_i(t) - A_i(t_0)\big) \;+\; \big(P_i(t) - P_i(t_0)\big).$$

PROOF. F_i changes only at dispatch and at completion. A dispatch of c adds $\hat{e}(c)/w_i$ to F_i and $\hat{e}(c)$ to P_i, leaving both sides equal. A completion of c adds $(a(c) - \hat{e}(c))/w_i$ to F_i, adds $a(c)$ to A_i, and subtracts $\hat{e}(c)$ from P_i; the right-hand side changes by $a(c) - \hat{e}(c)$, matching. Summing over all events in $(t_0, t]$ gives the identity. □

The identity says that an agent's virtual time is its reconciled service plus its outstanding phantom charge, in weighted units. A call in flight contributes its estimate; a call

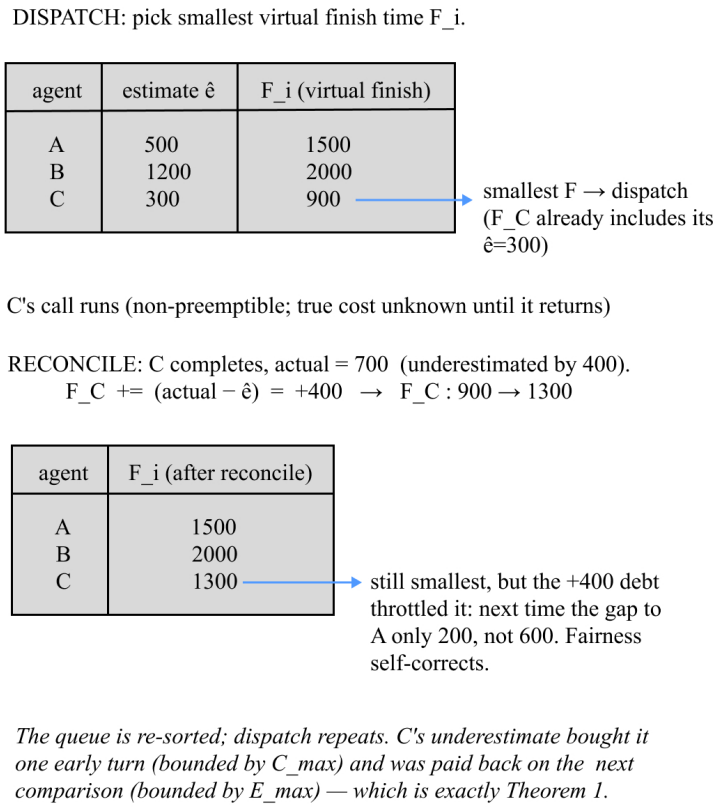

FIGURE 2. One RVT dispatch cycle. Three agents hold virtual times $F_A = 1500$, $F_B = 2000$, $F_C = 900$; the scheduler dispatches C, whose F already includes its estimate $\hat{e} = 300$. The call runs non-preemptibly and returns an actual cost of 700, an underestimate of 400. Reconciliation adds +400 to F_C, moving it to 1300: C is still the smallest and will run again, but its lead over A has narrowed from 600 to 200. The underestimate bought C one early turn and was repaid at the next comparison—which is the content of Theorem 1.

completed contributes its truth. This is the sense in which reconciliation "erases" the estimator: once a call completes, no trace of $\hat{e}(c)$ remains in F_i.

4.4. The fairness bound

Assumptions. (A1) Dispatch decisions are totally ordered by the kernel's single dispatch loop. (A2) Each agent has at most d calls in flight. (A3) Agents i and j are continuously backlogged over $[t_0, t]$, with $F_i(t_0) = F_j(t_0)$ and $P_i(t_0) = P_j(t_0) = 0$; that is, the period begins at an instant quiescent for both. (A4) $a(c) \le C_{\max}$ and $\hat{e}(c) \le \hat{E}_{\max}$. (A5) The admission predicate does not discriminate between i and j—both are admissible whenever the system dispatches.

Write $\Delta A_i = A_i(t) - A_i(t_0)$ for the service agent i receives during the period.

Algorithm 1 Reconciled Virtual Time

procedure SUBMIT(i, c)
  if $i \notin B$ **then**
    $F_i \leftarrow \max(F_i,\ \min_{j \in B} F_j)$
  end if
  $B \leftarrow B \cup \{i\}$
  $queue_i$.push(c)
  DISPATCH
end procedure

procedure DISPATCH
  $Cand \leftarrow \{i \in B : n_i < d \wedge queue_i \neq \varnothing \wedge \text{admissible}(i)\}$
  if $Cand = \varnothing$ **then**
    return
  end if
  $i \leftarrow \arg\min_{i \in Cand} F_i$ ▷ ties broken by agent id (I4)
  $c \leftarrow queue_i$.pop()
  $\hat{e} \leftarrow$ ESTIMATE(i, c)
  $F_i \leftarrow F_i + \hat{e}/w_i$
  $P_i \leftarrow P_i + \hat{e}$
  $n_i \leftarrow n_i + 1$
  log(DISPATCH, $i, c, \hat{e}$) ▷ I3, I4
  issue c to the provider
end procedure

procedure COMPLETE(i, c, a)
  $F_i \leftarrow F_i + (a - \hat{e}(c))/w_i$ ▷ RECONCILE
  $P_i \leftarrow P_i - \hat{e}(c)$
  $n_i \leftarrow n_i - 1$
  $A_i \leftarrow A_i + a$
  LEDGER.charge(i, a) ▷ I5
  log(COMPLETE, i, c, a)
  if $queue_i = \varnothing \wedge n_i = 0$ **then**
    $B \leftarrow B \setminus \{i\}$
  end if
  DISPATCH
end procedure

THEOREM 1 (RVT service-gap bound). *Under A1–A5, for all t in the backlogged period,*

$$\frac{\Delta A_i}{w_i} - \frac{\Delta A_j}{w_j} \;\le\; d\left(\frac{C_{\max}}{w_i} + \frac{\hat{E}_{\max}}{w_j}\right),$$

and symmetrically with i and j exchanged. For equal weights $w_i = w_j = w$,

$$\left|\frac{\Delta A_i}{w} - \frac{\Delta A_j}{w}\right| \;\le\; \frac{d\,(C_{\max} + \hat{E}_{\max})}{w}.$$

PROOF. Fix t and consider the direction stated. Two cases.

Case 1: agent i is dispatched at least once in $[t_0, t]$. Let τ be the instant of i's last dispatch at or before t, and let τ^- denote the instant immediately before that dispatch decision. Since the dispatch rule selects the candidate of least virtual time and j is backlogged and admissible at τ by A3 and A5,

$$F_i(\tau^-) \;\le\; F_j(\tau^-).$$

Apply Lemma 1 to each side at τ^-, using $P_i \ge 0$ and $F_i(t_0) = F_j(t_0)$:

$$\frac{\Delta A_i(\tau^-)}{w_i} \;\le\; F_i(\tau^-) - F_i(t_0) \;\le\; F_j(\tau^-) - F_j(t_0) \;=\; \frac{\Delta A_j(\tau^-)}{w_j} + \frac{P_j(\tau^-)}{w_j}.$$

By A2 and A4, $P_j(\tau^-) \le d\,\hat{E}_{\max}$. Since A_j is non-decreasing, $\Delta A_j(\tau^-) \le \Delta A_j(t)$. Hence

$$\frac{\Delta A_i(\tau^-)}{w_i} \;\le\; \frac{\Delta A_j(t)}{w_j} + \frac{d\,\hat{E}_{\max}}{w_j}.$$

It remains to bound i's service in $(\tau^-, t]$. Because τ is i's *last* dispatch in the window, the only calls of i that can complete in $(\tau^-, t]$ are those in flight at τ, of which there are at most d by A2. Each contributes at most $C_{\max}$, so $\Delta A_i(t) \le \Delta A_i(\tau^-) + d\,C_{\max}$. Combining,

$$\frac{\Delta A_i(t)}{w_i} \;\le\; \frac{\Delta A_j(t)}{w_j} + \frac{d\,\hat{E}_{\max}}{w_j} + \frac{d\,C_{\max}}{w_i},$$

which is the claim.

Case 2: agent i is never dispatched in $[t_0, t]$. By A3, $P_i(t_0) = 0$, so no call of i is in flight at t_0 and none is dispatched thereafter; hence $\Delta A_i(t) = 0$. Since $\Delta A_j(t) \ge 0$, the left-hand side is non-positive and the bound holds trivially.

The symmetric direction follows by exchanging i and j throughout. □

Reading the bound. The two terms map one-to-one onto the two ways this regime departs from classical WFQ. The $C_{\max}$ term is the **non-preemptibility gap**: having been selected fairly, an agent may complete calls the scheduler cannot recall, and their cost is discovered too late to prevent. The $\hat{E}_{\max}$ term is the **phantom-charge gap**: while a competitor holds a call in flight, its virtual time is inflated by an estimate that has not yet been reconciled, and the other agent may exploit that inflation to run ahead. Both are irreducible consequences of the interface, not artifacts of the algorithm—Proposition 3

shows the first is tight.

What the estimator does and does not buy. The bound depends on the estimator only through $\hat{E}_{\max}$—the *magnitude* of the in-flight charge—and not at all through its accuracy. This is a genuine and slightly counterintuitive consequence of reconciliation, and it revises the natural expectation that better output-length prediction buys better fairness. It does not. A perfect estimator and a systematically biased one yield the same bound if their charges have the same maximum, because reconciliation repairs the bias within one call. What a good estimator buys instead is (a) *admission safety*—the ability to refuse a call that would breach a rate-limit window or a dollar budget, which requires an upper bound on cost before the call is issued; (b) *lower transient unfairness within a dispatch round,* which is what a latency-sensitive user perceives even when the integrated gap is bounded; and (c) *tighter bounds at high fan-out,* since the $d\,\hat{E}_{\max}$ term grows with concurrency. We separate these roles explicitly because conflating them is what makes the naive design of Step 2 look reasonable.

4.5. Corollaries and negative results

COROLLARY 1 (Estimator-error form). *If the estimator never exceeds the true cost by more than $E_{\max}$—that is, $\hat{e}(c) \le a(c) + E_{\max}$ for all c—then $\hat{E}_{\max} \le C_{\max} + E_{\max}$, and for $d = 1$ and unit equal weights Theorem 1 gives*

$$\left|\Delta A_i - \Delta A_j\right| \;\le\; 2\,C_{\max} + E_{\max}.$$

This is the form in which the bound is most easily compared to VTC's 2× result (Sheng et al. 2024): the leading $2C_{\max}$ is the direct analogue of VTC's non-preemptibility constant, and $E_{\max}$ is the price of not knowing cost at dispatch. We present Theorem 1 in the $\hat{E}_{\max}$ form because it is tighter and because it is the form that exposes the estimator's actual role.

COROLLARY 2 (Starvation freedom). *Assume A1–A5 and $a(c) \ge c_{\min} > 0$. While a backlogged agent i waits without being dispatched, the aggregate service delivered to all other agents is at most*

$$\sum_{j \ne i} \left(d\,C_{\max} + \frac{w_j}{w_i}\, d\,\hat{E}_{\max} \right),$$

so at most $\sum_{j\ne i}\big(d\,C_{\max} + (w_j/w_i)\,d\,\hat{E}_{\max}\big)/c_{\min}$ calls of other agents can complete before i is dispatched. Agent i is therefore dispatched within bounded work.

PROOF. Apply Theorem 1 in the direction j over i, taking t_0 as the start of i's wait. Since i receives no service during the wait, $\Delta A_i = 0$, so $\Delta A_j / w_j \le d\,(C_{\max}/w_j + \hat{E}_{\max}/w_i)$, i.e. $\Delta A_j \le d\,C_{\max} + (w_j/w_i)\,d\,\hat{E}_{\max}$. Summing over $j \ne i$ bounds the aggregate; dividing by

$c_{\min}$ bounds the call count. □

Starvation freedom is thus not a separate mechanism but a consequence of the fairness bound—the desirable structure, since it means the two properties cannot drift apart in implementation. Priority classes complicate this and are treated in §5.2.

PROPOSITION 2 (Necessity of reconciliation). *Without the reconciliation step, the service gap is unbounded in time: for any G there is a workload and an instant at which $|\Delta A_1 - \Delta A_2| > G$.*

PROOF. Take two equal-weight agents, $d = 1$, both continuously backlogged, and a scheduler that advances F_i by $\hat{e}(c)$ at dispatch and never reconciles. Let agent 1's calls have $\hat{e} = \varepsilon$ and $a = C$; let agent 2's calls have $\hat{e} = a = C$. Agent 1's virtual time advances ε per call while agent 2's advances C, so between consecutive dispatches of agent 2 the scheduler dispatches agent 1 approximately C/ε times. Over m dispatches of agent 2, $\Delta A_1 \approx mC^2/\varepsilon$ while $\Delta A_2 = mC$, so the gap is $\approx mC(C/\varepsilon - 1)$, which grows without bound in m for any fixed $\varepsilon < C$. □

The proposition is what rules out the entire family of estimate-only designs, including the token-bucket and prediction-based schemes practitioners reach for first. Note that it is not a statement about bad estimators: agent 1's estimator need not be adversarial, merely optimistic, which is the empirically common case under heavy-tailed output lengths.

PROPOSITION 3 (Tightness of the $C_{\max}$ term). *There is a workload under which the gap reaches $C_{\max} - c_{\min}$, so the $C_{\max}$ term in Theorem 1 cannot be removed.*

PROOF. Two equal-weight agents at $F_1 = F_2 = 0$, $d = 1$. Agent 1's pending call has $a = C_{\max}$; agent 2's has $a = c_{\min}$. The scheduler breaks the tie in favor of agent 1 and dispatches it; the call is non-preemptible, so at its completion $\Delta A_1 = C_{\max}$ and $\Delta A_2 \le c_{\min}$. □

Any scheduler over non-preemptible quanta admits this instance, so the term is a property of the interface rather than of RVT. What RVT contributes is that this is *all* that remains, alongside the phantom-charge term the estimator controls.

4.6. Enforcement disciplines

Theorem 1 bounds unfairness given bounded charges. Two enforcement disciplines connect the bound to operational reality, and our analysis re-scopes both relative to how they are usually motivated.

Budget reservation. Charge the agent's declared `max_tokens` at dispatch and refund the unused portion on completion, so that $\hat{e}(c) \ge a(c)$ always. It is tempting to describe reservation as the mechanism that bounds unfairness. Theorem 1 shows it is not: reconciliation already does that, and reservation in fact *loosens* the bound, because

$\hat{E}_{\max}$ becomes the largest declared `max_tokens` rather than the largest expected cost. Reservation earns its place for a different reason. It is what makes admission control sound: because the charge is an upper bound, the kernel can guarantee that dispatching a call cannot overshoot a rate-limit window or a dollar budget, and the ledger never goes transiently negative (I5). The trade-off is explicit and worth stating in these terms—reservation buys budget safety at the cost of a looser fairness bound and briefly held-back siblings, and §7.2 measures where the empirical distribution sits between the two.

Debt carryover. When an agent's actual cost overruns its epoch budget, the overrun is carried into the next epoch as positive virtual-time debt rather than penalizing the current epoch retroactively. In RVT this is not an added mechanism: it is precisely the statement that F_i is never reset at epoch boundaries (§4.2). Its consequence for the analysis is that Theorem 1 applies across epoch boundaries unchanged, since the backlogged-period argument never refers to epochs. A runaway agent therefore cannot permanently degrade its siblings' service; its overrun is repaid on the next comparison, exactly as an underestimate is.

`cancel` as backstop. A call truncated mid-stream is charged for the tokens generated so far, and the remainder of the reservation is refunded into the next epoch. `cancel` is the only mid-call intervention available, and it forfeits rather than reclaims work, so it is a safety mechanism against pathological calls rather than a scheduling primitive. The bound holds with or without it.

4.7. Streaming calls

A streaming `infer` is treated as a single non-preemptible call for the purposes of the bound: the call's total cost remains unknown until the last chunk, so nothing in the analysis changes. `cancel` converts a streaming call into a truncated one whose cost is the tokens generated so far, which is the one genuine mid-call preemption point the whole-call model otherwise lacks. Re-deriving the bound for chunk-granular preemption—where the scheduler could in principle reclaim a call partway and the $C_{\max}$ term would shrink toward a per-chunk constant—is future analysis. This version claims the bound for the buffered case and measures streaming overhead separately (§7.3).

5. Beyond the Scalar Bound

5.1. Multi-resource fairness via DRF

Scalar virtual time presumes a single scarce resource. When agents bottleneck on different resources—one on tokens/min, another on requests/min, a third on dollars—scalar fair share is ill-defined, as §1.1 observed. We apply Dominant Resource Fairness (Ghodsi

et al. 2011) over the normalized vector

$$\left(\frac{\text{tok\_used}_i}{\text{tok\_limit}}, \frac{\text{req\_used}_i}{\text{req\_limit}}, \frac{\$\text{used}_i}{\$\text{limit}}\right).$$

Each agent's *dominant share* is the maximum of its normalized components. The scheduler minimizes the maximum dominant share across agents, and the per-agent virtual-time charge in Algorithm 1 is taken against the dominant dimension rather than against tokens. When one resource dominates for every agent, the vector collapses and the discipline degrades exactly to §4, recovering Theorem 1.

We treat this as a secondary result deliberately. The DRF theory transfers unchanged from (Ghodsi et al. 2011); the contribution here is the agent-specific resource model—what the dimensions are, how they are normalized against per-session configured limits rather than a universal token unit, and how they behave empirically on mixed-bottleneck workloads. We do not claim new fairness theory for the multi-resource case.

5.2. Priorities, aging, and priority inversion

Corollary 2 gives starvation freedom among equal-priority agents. Priority classes reintroduce the risk, since a persistently backlogged high-priority class can shut out a low-priority one. We adopt MLFQ-style classes (Corbato, Merwin-Daggett, and Daley 1962; Arpaci-Dusseau and Arpaci-Dusseau 2018) with aging: an agent's effective priority rises with waiting time, so every agent reaches the dispatchable class within a configurable bound of N epochs. Within a class, RVT governs; across classes, aging governs. Budget-exhausted agents move to `runnable` for the next epoch rather than blocking the queue, which preserves work conservation.

Preemption is cooperative at syscall boundaries: an agent is preemptible *between* calls, never mid-call, mirroring kernel preemption at syscall return in classical UNIX (Ritchie and Thompson 1974). Cross-agent waits can invert priorities—a high-priority agent blocked in `recv` on a low-priority sender inherits that sender's effective service rate. The kernel applies priority inheritance along the wait-for graph and detects deadlock cycles. This is available only because the kernel observes every `recv` wait (§3.2): it is a direct dividend of complete mediation, and a non-mediating layer cannot construct the graph at all (Proposition 1, P3).

5.3. Design alternatives

Virtual time is a choice, and each alternative fails on a property this regime demands.

Virtual time wins on three counts the others do not combine: it yields a closed-form service-gap bound, it is work-conserving, and—decisively—its backlogged-period argu-

TABLE 2. Design alternatives to virtual-time fair queueing

Alternative	Why not the substrate here
Lottery (Waldspurger 1994)	Fair only in expectation, with convergence requiring many quanta. LLM calls are few, coarse, and expensive, so per-run variance—not the mean—is what a user experiences. No closed-form per-run bound.
Stride (Waldspurger and Weihl 1995)	Deterministic proportional share, virtual time's discrete cousin. Its pass/stride increment assumes a *known* ticket cost per quantum, which is exactly what we lack. RVT keeps its virtual-time essence and adds reconciliation.
EDF / deadline	Agents are throughput-fair, not deadline-driven; there is no natural per-call deadline. EDF offers no proportional-share guarantee and degrades to arbitrary unfairness under overload.
MLFQ / CFS	Heuristic feedback with no closed-form fairness bound. We borrow MLFQ-style aging for priority classes (§5.2), but layered on a virtual-time core rather than as the core.
Token bucket per agent	Not work-conserving: an idle agent's unused rate is wasted, and there is no cross-agent proportionality when one agent is backlogged and another is not. It is the industry folk remedy, and a baseline in §7.2.

ment is the specific proof that survives the transplant. Lemma 1 and Theorem 1 are that argument, extended to absorb an unreconciled in-flight charge. We chose the substrate whose theorem we could carry across the break, not merely one that schedules.

6. Implementation

AgentKernel is implemented in Go [TBD: confirm language/LOC]. The kernel is an event loop with a single logical dispatch sequence (A1), which is simultaneously the scheduling serialization point, the logging point (I3), and the determinism point (I4). Its components correspond to the boxes in Figure 1:

Run queue and scheduler. Per-agent records holding $(F_i, w_i, P_i, n_i, A_i, \pi)$ in a priority structure keyed on F_i with deterministic tie-breaking by agent identifier. Dispatch is triggered on submission, on completion, and on rate-limit window refresh, so the scheduler never idles while an admissible backlogged agent exists.

Budget ledger. Per-agent and per-session accounting in tokens, dollars, and rate-limit slots, denominated per provider since tokenizers differ. Reservation charges and completion refunds pass through the ledger, and I5 is checked continuously against provider-reported usage.

Estimator. A per-agent EWMA over recent call costs, bootstrapped from a per-model prior, with the agent's declared `max_tokens` as an upper clamp. Under reservation the estimate *is* `max_tokens`. The estimator is kernel state and its version is logged, so dispatch decisions replay (I4).

Write-ahead log. Append-before-effect for every nondeterministic input and every

dispatch decision, used here for crash recovery: on restart the kernel replays from the last checkpoint, reconstructing run queue, ledger, and virtual times, and agents resume at their last syscall boundary.

Provider adapters. Each back-end is wrapped behind a uniform resource-vector interface exposing its rate-limit model, cost coefficients (α, β), and streaming semantics. Adding a provider does not change the scheduler.

Execution forms. The SDK path and the WASM sandbox path of §3.5, plus the compatibility shim used only for migration and measurement.

7. Evaluation

[TBD—This section states the experimental design and the hypotheses under test. Numbers are pending; every claim below is a hypothesis, not a result. Replace bracketed placeholders with measured values and add the result tables/figures before submission.]

7.1. Research questions and hypotheses

RQ0 (mediation tax). Can complete mediation be imposed at acceptable overhead?
H0a: Mediation overhead—scheduling, kernel dispatch, and WAL append—is under 5% of wall-clock and under 1% of token spend versus an unmediated LangGraph-style baseline on identical tasks.

RQ1 (scheduling). Does virtual-time fair scheduling of whole, non-preemptible LLM calls with unknown completion cost achieve bounded unfairness and starvation freedom at the runtime layer?
H1a: RVT keeps Jain's fairness index (Jain 1984) above 0.9 across 10 agents at 10:1 demand skew, versus below 0.6 for FCFS and `Promise.all`.
H1b: The maximum observed service gap between continuously backlogged equal-weight agents respects Theorem 1, and the bound is approached in the heavy-tailed adversarial case.
H1c: Under mixed priority classes with aging, no agent waits more than N epochs, with zero starvation events over sustained adversarial load.
H1d (secondary): The DRF extension identifies each agent's dominant bottleneck and achieves per-dimension fairness comparable to H1a when agents bottleneck on different resources.

7.2. Scheduler experiments

A deterministic mock-LLM load generator—latency and output-length distributions calibrated from real API traces, no spend—drives $n \in \{2, 10, 50\}$ agents under six workloads: uniform demand (baseline fairness); 10:1 demand skew (H1a); adversarial bursts, in which

one agent submits back-to-back maximum-cost calls; heavy-tailed output-length distributions (the estimator-stress case, and the setting of Proposition 2's construction); mixed priority classes with starvation-inducing low-priority agents (H1c); and a two-resource bottleneck with one agent tokens-limited and another requests-limited (H1d).

Metrics: Jain's index (Jain 1984) over sliding windows; maximum pairwise service gap against the Theorem 1 bound, reported as both worst case observed and full distribution; p50/p99 dispatch latency per priority class; starvation counts over 1000-epoch runs; and utilization as a work-conservation check.

Baselines: FCFS, unweighted round-robin, `Promise.all` (no scheduler), and token-bucket-per-agent (the folk remedy of §5.3). We additionally run **estimate-only RVT**—Algorithm 1 with the reconciliation line removed—as the ablation that instantiates Proposition 2 empirically, and **reservation on/off** to measure the fairness-versus-admission-safety trade-off identified in §4.6.

Because a bound stated at 100K-token scale is a liveness statement rather than a practical one, we report the proven worst case *and* the empirical distribution. The distribution is the operational claim; the bound is the guarantee.

7.3. Overhead and fidelity

Unmediated LangGraph/AutoGen baselines run against SDK-mediated and sandboxed configurations on identical GAIA (Mialon et al. 2023) and τ-bench (Yao et al. 2024) tasks at $n \in \{2, 10, 50\}$. The WASM sandbox's overhead and the kernel's dispatch-path latency are the two cost drivers and are reported separately. A kernel micro-benchmark measures syscall dispatch throughput under 50-agent concurrency, since serial dispatch is the candidate bottleneck and H0a must hold under load. Streaming overhead is measured separately per §4.7.

Small-scale live runs against two production providers validate mock fidelity—latency distributions, output-length distributions, and rate-limit behavior—before any mock-based claim is made.

7.4. Mediation leakage

The adversarial escape suite of §3.5 instantiates each Proposition 1 bypass concretely—direct sockets, subprocess spawns, second SDK instances, unmediated clock and RNG reads, out-of-band IPC—and measures I1 coverage under each execution form. We report which invariants survive under the compatibility shim, which is the practically important number for anyone migrating existing framework code, and expect the sandboxed form to close all five by construction.

8. Related Work

8.1. Agent operating systems

MemGPT (Packer et al. 2023) borrows virtual-memory language for context management and is the closest intellectual ancestor of the OS framing, though it addresses memory rather than scheduling. AIOS (Mei et al. 2025; Ge et al. 2023) is the closest system: it interposes a kernel with scheduler, context, and storage managers. On scheduling specifically, the delta is complete: AIOS schedules FIFO or round-robin, with no proportional-share guarantee, no proven unfairness bound, and no multi-resource accounting. The 2026 architectural analyses (Zhao, Johnson, and Gu 2026; Pirch, Goller, and Schlögl 2026; authors 2026a) are predominantly positional. The skeptical critique (Knowlee 2026)—that existing agent-OS work adopts the vocabulary of kernels while providing none of the guarantees, with no syscall interface and no concurrency model—we adopt as our evaluation bar rather than rebut; Proposition 1 and Theorem 1 are what meet it.

8.2. LLM serving systems

PagedAttention/vLLM (Kwon et al. 2023) operates at the KV-cache layer, invisible to application semantics. VTC (Sheng et al. 2024) proves a $2\times$ service bound for serving-engine scheduling with token-granular preemption via continuous batching (Yu et al. 2022); Sarathi-Serve (Agrawal et al. 2024) and Llumnix (Sun et al. 2024) refine intra-engine scheduling, and a 2026 line of work argues for a runtime layer between agent frameworks and serving engines (Zhang et al. 2026).

Our delta is the layer and, consequently, the problem. We operate above the provider API, where the scheduler sees entire calls across *multiple* providers and non-model tools (§1.2) rather than token streams inside one engine. Non-preemptibility and unknown cost define a different scheduling problem, and to our knowledge the fairness bound has not been established for it. The two layers are complementary rather than competing: engine-side output-length prediction would serve directly as an estimator hint in Algorithm 1, and by §4.4 its value would lie in admission safety and transient smoothness rather than in the fairness bound.

8.3. Cluster schedulers and multi-resource fairness

Borg (Verma et al. 2015), Kubernetes, and Ray (Moritz et al. 2018) schedule CPU, GPU, and memory; DRF (Ghodsi et al. 2011) supplies the multi-resource fairness theory we adapt to (tokens/min, requests/min, dollars/epoch). Token economics for agent systems is an emerging concern (authors 2026b). The theory transfers; the contribution here is the resource model, which is why §5.1 is a secondary result.

8.4. The unknown-cost scheduling problem

Classical proportional-share and fair-queueing disciplines—lottery and stride (Waldspurger 1994; Waldspurger and Weihl 1995), WFQ (Demers, Keshav, and Shenker 1989), SFQ (Goyal, Vin, and Cheng 1996)—assume the cost of a quantum is known when it is enqueued. The regime we face, where cost is revealed only at completion and the quantum cannot be preempted to reclaim an overrun, is precisely the case those analyses set aside. The nearest classical intuition is packet scheduling under unknown packet size over a constrained link, but without the router's option of dropping or fragmenting. Reconciliation is our substitute for the preemption-on-overrun that the classical setting takes for granted, and Proposition 2 shows some such substitute is required.

9. Threats to Validity and Translation Residue

Where the OS translation breaks, we characterize the break rather than footnote it.

No hardware preemption. Classical proportional-share schedulers obtain preemption from the timer interrupt. We obtain `cancel`, which is cooperative: an agent that refuses to emit syscalls cannot be preempted. For sandboxed agents the WASM fuel limit is the analogue of a hardware interrupt; for SDK agents the claim is precisely "preemptible at syscall boundaries, assuming the agent eventually calls a syscall." This is stated as an assumption, not assumed away.

Estimator adversaries. The $\hat{E}_{\max}$ term is bounded only if the kernel bounds the charge. An agent that declares `max_tokens` = 1 while consuming 100K is charged one token's worth at dispatch, so it is dispatched more often than it should be—but reconciliation repays the debt at completion, so the effect appears as dispatch-latency advantage rather than as a fairness violation, and Theorem 1 continues to hold with $\hat{E}_{\max}$ small. The honest residue is that transient latency advantage is real and is not what the theorem bounds; we measure it with an adversarial agent (§7.2).

Assumption A5 (non-discriminatory admission). The bound assumes provider rate limits do not systematically exclude one agent while admitting another. Cross-provider deployments can violate this: an agent bound to a provider whose window is exhausted is inadmissible through no fault of the scheduler. In that regime fairness is bounded per provider pool, not globally, and we report the per-pool decomposition.

Multi-resource normalization is provider-relative. Rate limits and costs differ across providers, and tokenizers differ, so the DRF extension normalizes against per-session configured limits rather than a universal token unit. Cross-provider accounting drift is measured and reported.

Baseline realism. Mock-LLM fidelity is validated against real API traces before any scheduler claim is made, and all primary metrics have mechanical ground truth—no LLM

judges.

Scope. Single node; no distributed scheduling; no kernel hot standby, crash recovery being WAL replay. Deterministic replay from the log, for debugging or audit, is left to future work.

10. Conclusion

Multi-agent LLM systems have concurrency without a concurrency policy, and no provider is positioned to supply one: the contention is visible only at the runtime the agents share. We argued that the process, not the chat session, is the abstraction under which a policy is statable, and built the scheduler that abstraction makes possible.

Reconciled Virtual Time extends virtual-time fair queueing to quanta that are non-preemptible and whose cost is unknown until completion—by charging an estimate at dispatch and reconciling the truth at completion. The resulting service gap is bounded by $d\,(C_{\max}/w_i + \hat{E}_{\max}/w_j)$, with each term traceable to one way the regime departs from the classical setting: the call the scheduler cannot recall, and the estimate it has not yet corrected. Two negative results fix the boundaries of the design space: without reconciliation the gap grows without bound, and without complete mediation the premises cannot be enforced at all. The analysis also revises a natural intuition—reconciliation, not prediction accuracy, is what buys fairness; the estimator's real job is admission safety.

The natural next questions are chunk-granular preemption, which would shrink the $C_{\max}$ term toward a per-chunk constant and requires re-deriving the bound; distributed scheduling across kernel instances sharing a provider pool; and co-design with serving-side schedulers, where engine-level output-length prediction and client-level reconciliation are complementary rather than redundant.

References

Agrawal, Amey, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2024. "Sarathi-Serve: Efficient LLM Serving with Heterogeneous LLM Architectures." In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Arpaci-Dusseau, Remzi H. and Andrea C. Arpaci-Dusseau. 2018. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books.

authors, Various. 2026a. "Runtime Infrastructure for Agentic Workloads." Workshop report.

authors, Various. 2026b. "Token Economics for Agent Systems." Workshop on Agent Economics, ICML.

Corbato, Fernando J., Marjorie Merwin-Daggett, and Robert C. Daley. 1962. "An Experimental Time-Sharing System." In *Proceedings of the AFIPS Spring Joint Computer Conference*.

CrewAI. 2024. "CrewAI: Framework for Orchestrating Autonomous AI Agents." `https://github.com/crewAIInc/crewAI`.

Demers, Alan, Srinivasan Keshav, and Scott Shenker. 1989. "Analysis and Simulation of a Fair Queueing Algorithm." In *Proceedings of ACM SIGCOMM*.

Dennis, Jack B. and Earl C. Van Horn. 1966. "Programming Semantics for Multiprogrammed Computations." *Communications of the ACM* 9 (3): 143–155.

Ge, Yingqiang, Yujie Ren, Wenyue Hua, Shuyuan Xu, Juntao Tan, and Yongfeng Zhang. 2023. "AIOS: An Operating System for AI Agents." *arXiv preprint arXiv:2311.15381*.

Ghodsi, Ali, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. 2011. "Dominant Resource Fairness: Fair Allocation of Multiple Resource Types." In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Goyal, Pawan, Harrick M. Vin, and Haichen Cheng. 1996. "Start-Time Fair Queueing: A Scheduling Algorithm for Integrated Services Packet Switching Networks." In *Proceedings of ACM SIGCOMM*.

Haas, Andreas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. 2017. "Bringing the Web up to Speed with WebAssembly." In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

Hong, Sirui, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2024. "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework." *arXiv preprint arXiv:2308.00352*.

Jain, Raj. 1984. *The Art of Computer Systems Performance Analysis*. Wiley.

Knowlee, J. 2026. "The Agent OS Mirage: Why Existing Agent Systems Are Not Operating Systems." Blog post.

Kwon, Woosuk, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. "Efficient Memory Management for Large Language Model Serving with PagedAttention." In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*.

LangChain. 2024. "LangGraph: Building Stateful, Multi-Actor Applications with LLMs." `https://github.com/langchain-ai/langgraph`.

Li, Guohao, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. "CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society." *Advances in Neural Information Processing Systems (NeurIPS)* 36.

Mei, Kai, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2025. "AIOS: LLM Agent Operating System." *arXiv preprint arXiv:2403.16971*.

Mialon, Grégoire, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and David Lopez-Paz. 2023. "GAIA: A Benchmark for General AI Assistants." In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Miller, Mark S. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Johns Hopkins University Press.

Mohan, C., Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. "ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging." *ACM Transactions on Database Systems* 17 (1): 94–162.

Moritz, Philipp, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, et al. 2018. "Ray: A Distributed Framework for Emerging AI Applications." In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Packer, Charles, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. "MemGPT: Towards LLMs as Operating Systems." In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Pirch, Lukas, Sven Goller, and Alexander Schlögl. 2026. "Architectures for Multi-Agent Systems: A Survey." ArXiv preprint.

Ritchie, Dennis M. and Ken Thompson. 1974. "The UNIX Time-Sharing System." *Communications of the ACM* 17 (7): 365–375.

Saltzer, Jerome H. and Michael D. Schroeder. 1975. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63 (9): 1278–1308.

Sheng, Ying, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Zhuo Feng, Ion Stoica, and Joseph E. Gonzalez. 2024. "VTC: Fair Scheduling for LLM Serving." In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Sun, Bohan, Zizhao Mo, Xupeng Miao, Jianfei Chen, and Zhihao Jia. 2024. "Llumnix: Dynamic Scheduling for Large Language Model Serving." In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Verma, Abhishek, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. "Large-Scale Cluster Management at Google with Borg." In *Proceedings of the 10th European Conference on Computer Systems (EuroSys)*.

Waldspurger, Carl A. 1994. *Lottery and Stride Scheduling: Flexible Proportional-Share Resource Management*. PhD thesis, Massachusetts Institute of Technology.

Waldspurger, Carl A. and William E. Weihl. 1995. "Stride Scheduling: Deterministic Proportional-Share Resource Management." *Technical Report MIT/LCS/TM-528*.

Wu, Qingyun, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. "AutoGen: Enabling Next-Generation LLM Applications via Multi-Agent Conversation." In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Yao, Shunyu, Noah Shinn, Arjun Guha, Karthik Narasimhan, and John Peurifoy. 2024. "τ-bench: A Benchmark for Tool-Augmented Agents." In *Proceedings of the International Conference on Machine Learning (ICML)*.

Yu, Gyeong-In, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. "Orca: A Distributed Serving System for Transformer-Based Generative Models." In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Zhang, Mingyuan et al. 2026. "A Runtime Layer for Agentic AI Systems." ArXiv preprint.

Zhao, Andrew, Daniel D. Johnson, and Shixiang Shane Gu. 2026. "Agent Operating Systems: An Architectural Analysis." ArXiv preprint.

Appendix A. An Alternative Derivation of Lemma 1

The accounting identity admits a continuous-time interpretation. Let $\delta_i(t)$ be a Dirac comb at dispatch and completion events for agent i. Then

$$\frac{d}{dt}\big(w_i F_i(t)\big) = \dot{A}_i(t) + \dot{P}_i(t),$$

where $\dot{A}_i$ counts completed costs and $\dot{P}_i$ counts dispatch estimates minus completed charges. Integrating from t_0 to t recovers Lemma 1. The discrete proof in the main text is equivalent but more direct for the event-driven setting.

Appendix B. Provider Adapter Interface

The kernel abstracts each provider behind a uniform resource-vector interface:

- `Provider.Delegate(call)` — issue a call and return a future.
- `Provider.Cancel(call)` — attempt to truncate an in-flight call.
- `Provider.RateLimit()` — current remaining tokens/min, requests/min.
- `Provider.CostModel()` — the coefficients (α, β) for the provider's pricing.

New providers implement this interface; the scheduler and ledger remain unchanged. This is standard adapter-pattern engineering, stated here to fix the implementation boundary that the evaluation (§7) measures across.

Appendix C. Deadlock Detection via the Wait-For Graph

Because the kernel observes every `recv` syscall, it maintains the inter-agent wait-for graph $G = (V, E)$ where V is the set of agents and $(i, j) \in E$ iff agent i is blocked in `recv` waiting for agent j to `send`. A cycle in G is a deadlock. The kernel detects cycles on each blocking event by a depth-first search of G, which is $O(|V| + |E|)$ per event and negligible at single-node scale. On detection, the kernel may terminate the youngest agent in the cycle or escalate to a supervisor agent; the policy is configurable.

This is a direct dividend of complete mediation (I1). No non-mediating layer can construct G without observing every `send`/`recv` pair, which Proposition 1 shows a non-mediating layer cannot guarantee.